

1. Shashwathks Python For Automation And Ai Agents Assignment1

A company allocates employees into 3 shifts (Morning, Evening and Night) based on workload.

- You are given number of employees (N) and total tasks to be completed (T).
- Each employee can handle a maximum of 5 tasks per day (assume uniform capacity).
- Allocate employees to shifts dynamically so that tasks are completed with minimum overtime.
- If at any point total capacity < total tasks, print “Additional Resources Required
- If shifts overloaded (>10 employees per shift), trigger “Re-distribution Recommended”.

```
# Import required libraries (none needed in this case)

# Get total number of employees from user input
total_employees = int(input("Enter total employees :"))

# Get total number of tasks from user input
total_tasks = int(input("Enter total tasks: "))

# Initialize variables for morning, evening, and night shifts
morning_shift = evening_shift = night_shift = 0

# Calculate total capacity based on total number of employees and 5 tasks per employee
total_capacity = total_employees * 5

# Print output message
print("Output:")

# Check if total capacity is greater than total tasks
if (total_capacity > total_tasks):
    # Initialize a flag to track if any employee was assigned a shift
    assigned_shift = False

    # Loop through each employee
    for i in range(total_employees):
        # Check if morning shift is available
        if (morning_shift < 10):
            # Increment morning shift by 1 and decrement total employees by 1
            morning_shift += 1
            total_employees -= 1
            # Set flag to True indicating an employee was assigned a shift
            assigned_shift = True
        # Check if evening shift is available
        elif (evening_shift < 10):
            # Increment evening shift by 1 and decrement total employees by 1
            evening_shift += 1
            total_employees -= 1
            # Set flag to True indicating an employee was assigned a shift
            assigned_shift = True
        # Check if night shift is available
        elif (night_shift < 10):
            # Increment night shift by 1 and decrement total employees by 1
```

```

        night_shift += 1
        total_employees -= 1
        # Set flag to True indicating an employee was assigned a shift
        assigned_shift = True

    # Print number of employees assigned to each shift
    if assigned_shift:
        print(f"Morning Shift: {morning_shift} employees\nEvening Shift:
{evening_shift} employees\nNight Shift :{night_shift} employees")
    else:
        print("No employees assigned to any shift")
else:
    # Print message indicating additional resources are required
    print("Additional Resources required")

# Print total capacity
print(f"\nTotal Capacity : {total_capacity} tasks")

# Check if total capacity is greater than or equal to total tasks
if (total_capacity >= total_tasks):
    # Print message indicating tasks can be completed without overtime
    print("Status : Tasks can be completed without any overtime")
else:
    # Print message indicating overtime is required
    print("Overtime required")

```

Output Block

Output:
Morning Shift: 10 employees
Evening Shift: 10 employees
Night Shift :8 employees

Total Capacity : 140 tasks
Status : Tasks can be completed without any overtime

```

applicants = int(input("Enter the number of applicants :"))
// Initialize an empty dictionary to store applicant information
applicants_list = dict()
// Loop to collect information for each applicant
for i in range(applicants):
    credit_score = int(input("Credit Score: "))
    // Get the credit score for the current applicant
    income = int(input("Income : "))
    // Get the income for the current applicant
    applicants_list[i] = {"Credit_Score":credit_score,"Income":income}
// Initialize an array of application statuses with the same length as the number of
applicants
application_status = [] * len(applicants_list)
// Initialize a flag to track if an unusual pattern has been detected
flag = 0
// Initialize a counter for rejected applications
reject_count = 0
// Loop to evaluate each applicant's application status
for i in range(len(applicants_list)):
    // Check if the applicant's credit score is 750 or higher and their income is

```

```

800,000 or higher
    if(applicants_list[i]["Credit_Score"]>=750 and
applicants_list[i]["Income"]>=800000):
        application_status.append("Approved")
        // Reset the reject count for this applicant
        reject_count = 0
        // Check if the applicant's credit score is between 650 and 749 and their income
is 1,000,000 or higher
        elif(applicants_list[i]["Credit_Score"] in range(650,750) and
applicants_list[i]["Income"]>=1000000):
            application_status.append("Manual Review")
            // Reset the reject count for this applicant
            reject_count = 0
            // Check if the applicant's credit score is below 650
            elif(applicants_list[i]["Credit_Score"]<650):
                application_status.append("Rejected")
                // Increment the reject count
                reject_count+=1
                // Check if the reject count has reached 3
                if(reject_count >=3):
                    flag = 1
// Print the application statuses for all applicants
print("Output")
for i in range(len(application_status)):
    print(f"Application {i+1} : {application_status[i]}")
// Check if an unusual pattern has been detected
if(flag == 1):
    print("\nAudit Alert : Unusual pattern detected")

```

Output Block

```

Output
Application 1 : Rejected
Application 2 : Rejected
Application 3 : Rejected
Application 4 : Approved
Application 5 : Rejected
Application 6 : Approved
Audit Alert : Unusual pattern detected

```

```

// Get the number of data centers from user input
total_datacenters = int(input("Enter number of data centers: "))

// Define the total hours in a day
total_hours = 24

// Initialize an empty list to store data center information
data_center = []

// Initialize an empty list to store average usage percentages
average_usage = []

// Loop through each data center
for i in range(total_datacenters):
    // Get the data center information for the current iteration
    data_center.append([int(input()) for j in range(total_hours)])

```

```

// Print a title for the report
print(f"-----\n AI BASED ENERGY MONITORING\nREPORT\n-----")

// Loop through each data center
for i in range(total_datacenters):
    // Print the current data center number
    print(f>Data Center {i+1}<

    // Calculate the average usage percentage for the current data center
    average_usage.append(sum(data_center[i])/total_hours)

    // Print the average usage percentage
    print(f>Average Usage : {round(average_usage[i],2)}%<

    // Check if the average usage is above 90% and the current hour's usage is above
    95%
    for j in range(total_hours):
        if(average_usage[i] > 90 and data_center[i][j] > 95):
            // Print a critical overload message
            print(f>Critical Overload detected at Hour {j+1} (Usage :
{data_center[i][j]}%<)
            break

    // Check if the average usage is between 80 and 90 (inclusive)
    if(average_usage[i] in range(80,90)):
        // Print a hardware upgrade recommendation message
        print(f>Hardware upgrade recommended<

    // Check if the average usage is below 80
    if(average_usage[i] < 80):
        // Print a no critical overload message
        print(f>No Critical overload detected<

// Find the minimum average usage
min_data_usage = min(average_usage)

// Print the most efficient data center information
print(f>\n-----\nMost Efficient Data Center : Data
Center {average_usage.index(min(average_usage))+1}\n(Lowest Avg Energy
Usage:{round(min_data_usage,2)}%<)
print("-----")

// Print a conclusion message
print(>Conclusion:<)
// Loop through each data center
for i in range(total_datacenters):
    // Check if the average usage is below 80
    if(average_usage[i] < 80):
        // Print a normal operation message
        print(f>Data Center {i+1} performing normally<
    else:
        // Print a performance optimization and load balancing message
        print(f>Data Center {i+1} requires performance optimization and load
balancing.<)

```

Output Block

AI BASED ENERGY MONITORING REPORT

Data Center 1

Average Usage : 75.75%

No Critical overload detected

Data Center 2

Average Usage : 59.17%

No Critical overload detected

Data Center 3

Average Usage : 92.17%

Critical Overload detected at Hour 8 (Usage : 97%)

Most Efficient Data Center : Data Center 2

(Lowest Avg Energy Usage:59.17%)

Conclusion:

Data Center 1 performing normally

Data Center 2 performing normally

Data Center 3 requires performance optimization and load balancing.

```
#total_datacenters = int(input("Enter number of data centers: "))
#total_hours = 24
#data_center = [[72, 78, 80, 88, 90, 85, 70, 65, 60, 58, 62, 64, 68, 75, 66, 71, 73,
76, 80, 82, 85, 88, 90, 92],
#               [68, 70, 65, 60, 58, 64, 62, 61, 59, 57, 56, 55, 54, 53, 52, 51, 55,
58, 60, 62, 63, 61, 59, 57],
#               [75, 78, 80, 85, 90, 95, 92, 97, 99, 100, 105, 110, 108, 106, 102, 98,
95, 92, 90, 88, 86, 83, 80, 78]]
#average_usage = []
# print(f"-----\n AI BASED ENERGY MONITORING
REPORT\n-----")
for i in range(3):
    # Print header for each data center
    print(f"\nData Center {i+1}")
    # Calculate average usage for the current data center and append to the list
    average_usage.append(sum(data_center[i])/total_hours)
    # Print the average usage for the current data center
    print(f"Average Usage : {round(average_usage[i],2)}%")
    # Iterate over each hour of the data center
    for j in range(total_hours):
        # Check if the average usage is greater than 90 and the current hour's usage
        # is greater than 95
        if(average_usage[i] > 90 and data_center[i][j] > 95):
            # Print a critical overload detection message and break the loop
            print(f"Critical Overload detected at Hour {j+1} (Usage :
{data_center[i][j]}%)")
            break
        # Check if the average usage is between 80 and 90 (inclusive)
        if(average_usage[i] in range(80,90)):
            # Print a hardware upgrade recommendation message
            print(f"Hardware upgrade recommended")
        # Check if the average usage is less than 80
        if(average_usage[i] < 80):
            # Print a no critical overload detected message
            print(f"No Critical overload detected")
```

```

# Find the minimum average usage across all data centers
min_data_usage = min(average_usage)
# Print the most efficient data center and its average usage
print(f"\n-----\nMost Efficient Data Center : Data
Center {average_usage.index(min(average_usage))+1}\n(Lowest Avg Energy Usage:
{round(min_data_usage,2)}%)")
# Print the conclusion
print("-----")
# Print a conclusion message
print("Conclusion:")
for i in range(3):
    # Check if the average usage is less than 80
    if(average_usage[i]<80):
        # Print a normal performance message for the current data center
        print(f"Data Center {i+1} performing normally")
    else:
        # Print a performance optimization and load balancing recommendation message
        for the current data center
        print(f"Data Center {i+1} requires performance optimization and load
balancing.")

```

Output Block

```

-----
AI BASED ENERGY MONITORING REPORT
-----

Data Center 1
Average Usage : 75.75%
No Critical overload detected

Data Center 2
Average Usage : 59.17%
No Critical overload detected

Data Center 3
Average Usage : 92.17%
Critical Overload detected at Hour 8 (Usage : 97%)

-----
Most Efficient Data Center : Data Center 2
(Lowest Avg Energy Usage: 59.17%)
-----

Conclusion:
Data Center 1 performing normally
Data Center 2 performing normally
Data Center 3 requires performance optimization and load balancing.

```

```

# Initialize variables
fail_trigger = fail_count = 0 # Counter for total failures and current count of
failures
curr_count = 0 # Current count of login attempts

# Main loop to track login attempts until 5 consecutive failures
while(fail_count != 5):
    # Get user input for login attempt

```

```

login_attempt = input()
# Increment current attempt count
curr_count += 1
# Check if the current attempt is a failure
if fail_count < 5:
    # If the login attempt is a failure, increment the total failure count
    if(login_attempt == "FAILED"):
        fail_count += 1
    # If the login attempt is a success, reset the total failure count
    elif(login_attempt == "SUCCESS"):
        fail_count = 0

# Print total attempts and account lockdown message
print(f"Total Attempts : {curr_count}")
print("Account lockdown triggered due to 5 consecutive failures")

```

Output Block

```

Total Attempts : 5
Account lockdown triggered due to 5 consecutive failures

```

```

total_nurses_available = int(input("Total nurses available : "))
patients_severity = [int(input()) for i in range(total_nurses_available)]
n = len(patients_severity)
assigned_nurses = [] * n
print("Output\n")
// Iterate through the patients' severity levels and assign nurses accordingly
for i in range(n):
    if(patients_severity[i] >= 8):
        // If the patient's severity is 8 or above, assign 2 nurses
        assigned_nurses.append(2)
    elif(patients_severity[i] in range(5,8)):
        // If the patient's severity is between 5 and 8, assign 1 nurse
        assigned_nurses.append(1)
    elif(patients_severity[i] < 5):
        // If the patient's severity is below 5, assign 0 nurses
        assigned_nurses.append(0)
// Print the assigned nurse count for each patient
for i in range(n):
    print(f"Patient {i+1} -> {assigned_nurses[i]} nurse")
// Calculate the total assigned nurses
print(f"\nTotal assigned : {sum(assigned_nurses)} -> ", end="")
// Check if the total assigned nurses exceed the available nurses
if(sum(assigned_nurses) > total_nurses_available):
    // If so, recommend reallocation
    print("Shift Reallocation Recommended")
// Check if the total assigned nurses are less than the available nurses
elif(sum(assigned_nurses) < total_nurses_available):
    // If so, print the number of nurses available for reallocation
    print(f"{total_nurses_available-sum(assigned_nurses)} nurses available")
// If the total assigned nurses match the available nurses, print a success message
else:
    print("All nurses allocated, No Reallocation needed")

```

Output Block

Output

Patient 1 -> 2 nurse

Patient 2 -> 1 nurse

Patient 3 -> 2 nurse

Patient 4 -> 0 nurse

Total assigned : 5 -> Shift Reallocation Recommended

1. Linear Regression

Linear Regression

Source : Google Developers

Linear regression is a statistical technique used to find the relationship between variables. In an ML context, linear regression finds the relationship between features and a label.

In other words

Linear regression attempts to model the relationship between two variables by fitting a linear equation to observed data. One variable is considered to be an explanatory variable, and the other is considered to be a dependent variable. For example, a modeler might want to relate the weights of individuals to their heights using a linear regression model.

For example, suppose we want to predict a car's fuel efficiency in miles per gallon based on how heavy the car is, and we have the following dataset:

Pounds in 1000s (feature)	Miles per gallon (label)
3.5	18
3.69	15
3.44	18
3.43	16
4.34	15
4.42	14
2.37	24

Following program demonstrates the values mentioned in the above table and gives a visual representation using scatter plot.

```
import matplotlib.pyplot as plt

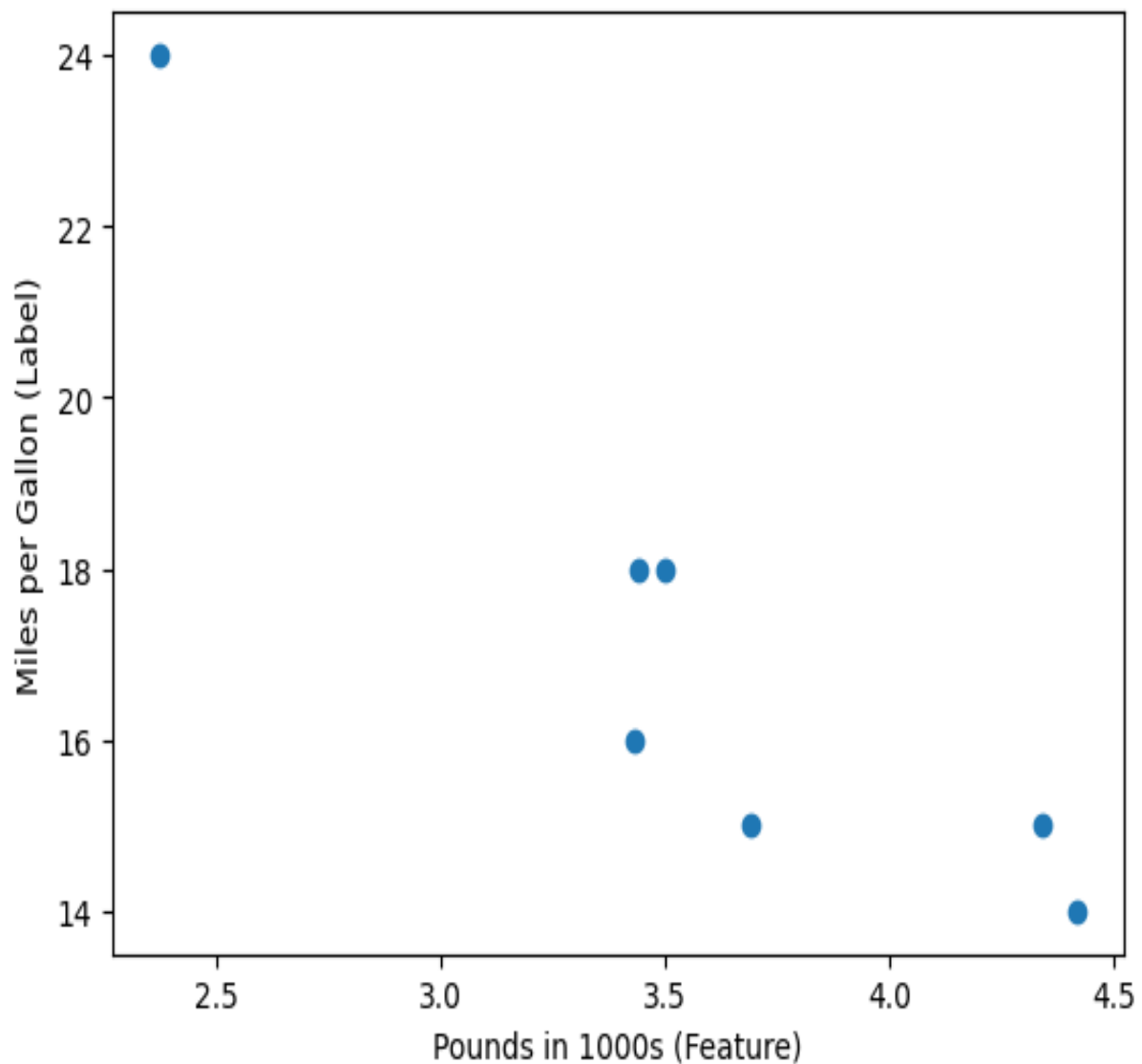
# Define the x and y coordinates
x = [3.5, 3.69, 3.44, 3.43, 4.34, 4.42, 2.37]
y = [18, 15, 18, 16, 15, 14, 24]

# Create a scatter plot
plt.scatter(x,y)

# Set the labels for the x and y axes
plt.xlabel("Pounds in 1000s (Feature)")
plt.ylabel("Miles per Gallon (Label)")
```

```
# Display the plot  
plt.show()
```

Output Block



#2. Using pyplot and scipy (built-in libs) to perform linear regression using scatter-plot and stats [from scipy].

```
import matplotlib.pyplot as plt  
from scipy import stats  
  
// Define the input data  
x = [3.5, 3.69, 3.44, 3.43, 4.34, 4.42, 2.37]  
y = [18, 15, 18, 16, 15, 14, 24]  
  
// Perform linear regression on the data  
// This function returns the slope, intercept, r-value, p-value, and standard error of
```

```
the regression line
slope, intercept, r_value, p_value, std_error = stats.linregress(x, y)

// Define a function to make predictions based on the linear regression model
// This function takes a value as input and returns the predicted value
def predict(x):
    // Use the linear regression model to make a prediction
    return slope * x + intercept

// Use the predict function to create a list of predicted values for each input value
predict_value = list(map(predict, x))

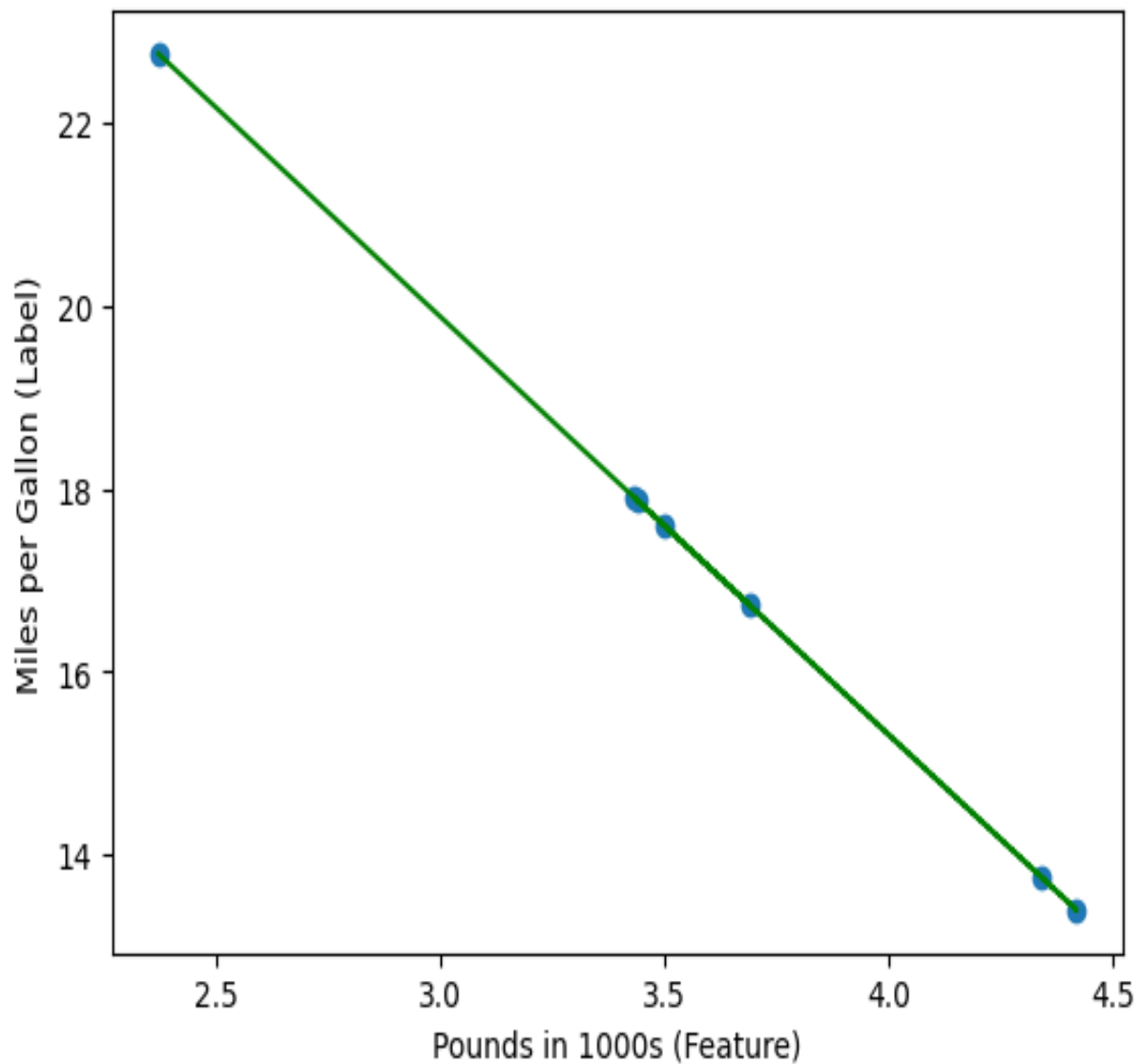
// Create a scatter plot of the input data
plt.scatter(x, predict_value)

// Plot the regression line on the scatter plot
plt.plot(x, predict_value, color='green')

// Set the labels for the x and y axes
plt.xlabel("Pounds in 1000s (Feature)")
plt.ylabel("Miles per Gallon (Label)")

// Display the plot
plt.show()
```

Output Block



#3. Using RAW approach with the step-by-step iteration of the linear regression methods

```
import matplotlib.pyplot as plt

class Solution:
    # Calculate the equation for linear regression
    def calculate_equation(self, x, y):
        # Check if the number of independent and dependent variables are the same
        n = len(x)
        n1 = len(y)
        if n != n1:
            return "The number of independent and dependent variables must be the same"

        # Calculate the x^2, y^2 and x*y for each value and store the items in a list
        x_squared = [i**2 for i in x]
        y_squared = [i**2 for i in y]
```

```

xy = [x[i]*y[i] for i in range(n)]

# Calculate the sum of x, y, x^2, y^2 and x*y
sum_x = sum(x)
sum_y = sum(y)
sum_x_squared = sum(x_squared)
sum_y_squared = sum(y_squared)
sum_xy = sum(xy)

# Calculate the slope (m) and intercept (b) using the formulas
m = (n*sum_xy - sum_x*sum_y) / (n*sum_x_squared - sum_x**2)
b = (sum_y - m*sum_x) / n

# Return the slope and intercept as values
return m, b

# Plot the linear regression line
def linear_regression(self, x, y):
    # Calculate the equation using the calculate_equation method
    m, b = self.calculate_equation(x, y)

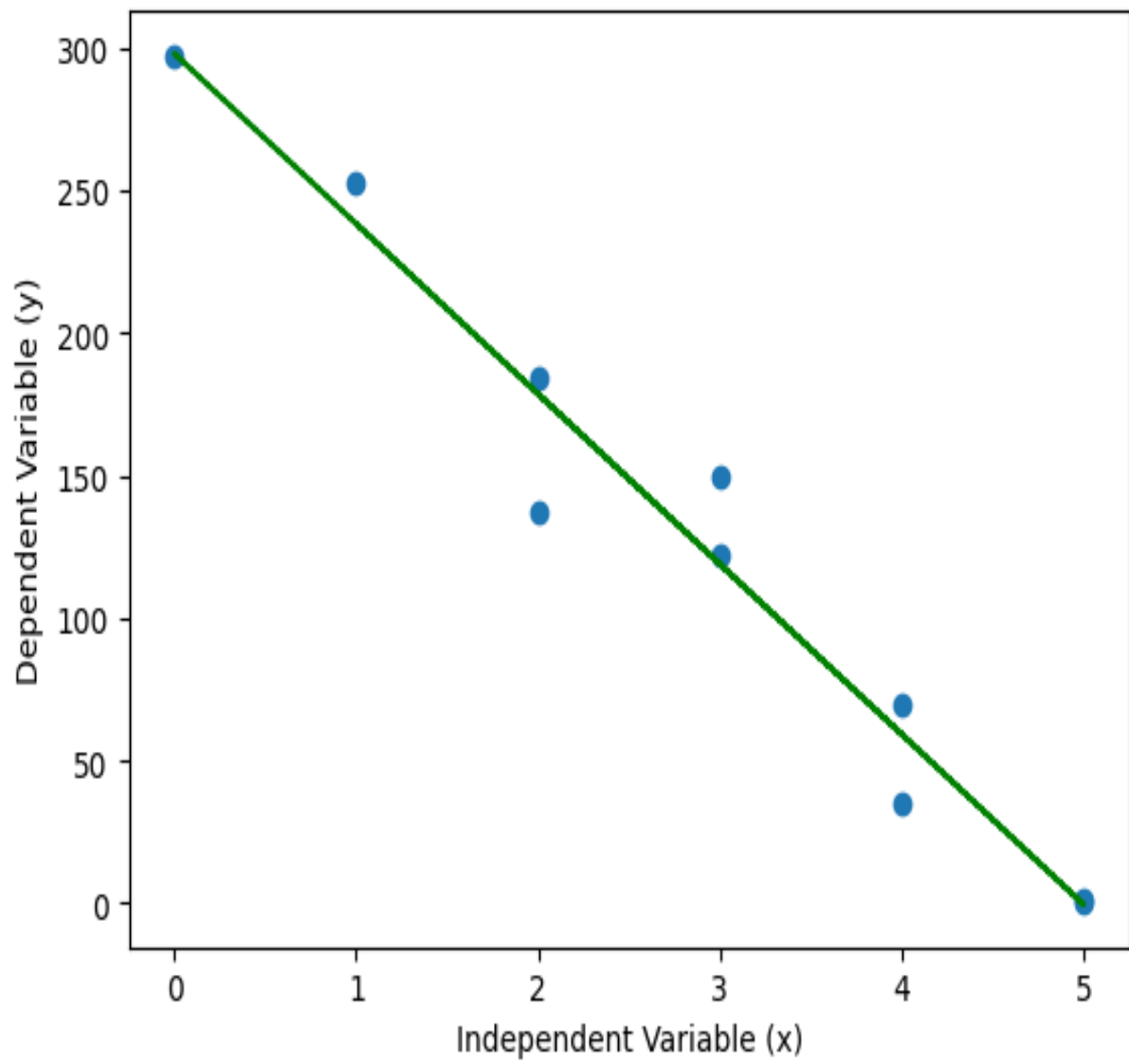
    # Create a scatter plot of the data
    plt.scatter(x,y)
    # Plot the linear regression line
    plt.plot(x, [m*i + b for i in x], color='green')
    # Set the labels for the x and y axes
    plt.xlabel("Independent Variable (x)")
    plt.ylabel("Dependent Variable (y)")
    # Display the plot
    plt.show()

if __name__ == "__main__":
    # Input the variables
    # Decide which is the Independent variable (x) and which is the Dependent variable
    (y)
    x = [2,4,2,5,4,0,3,5,1,3]
    y = [137, 70, 184, 0, 35, 297, 122 , 1, 253, 150]

    # Create an instance of the Solution class
    solution = Solution()
    # Call the linear_regression method
    result = solution.linear_regression(x,y)

```

Output Block



None